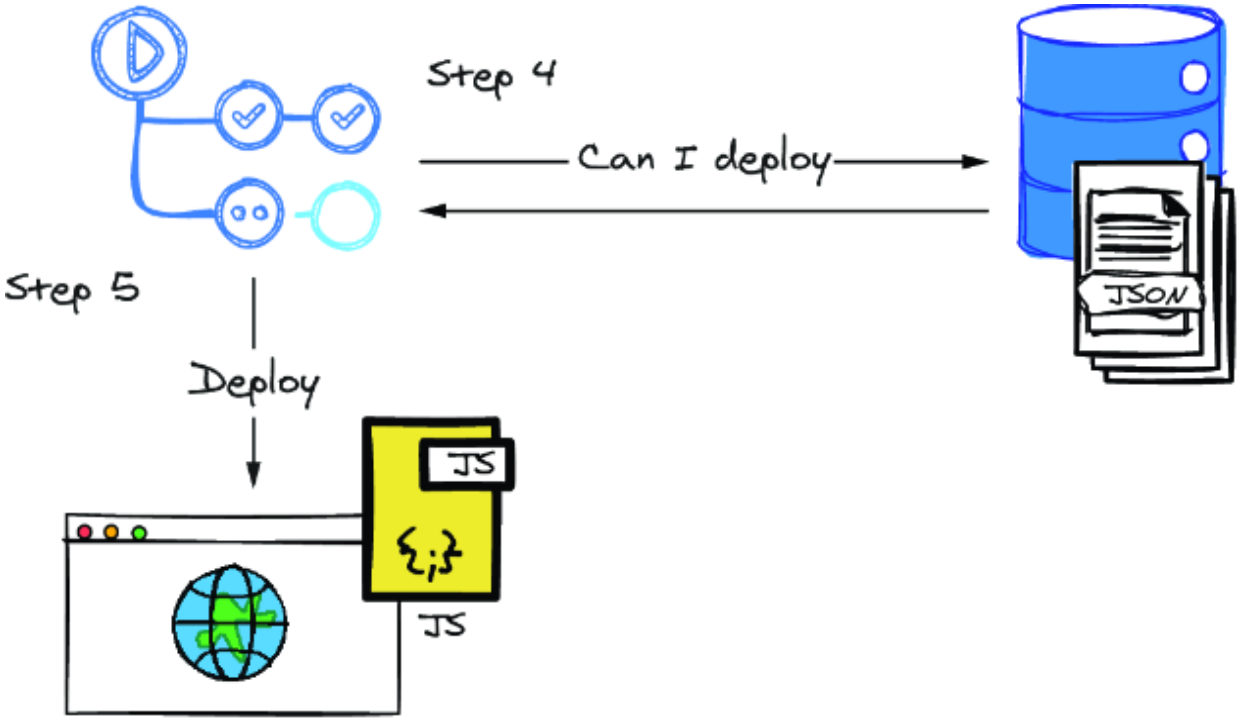


Figure 3.8 The consumer CI pipeline checks the contract status in the broker and can deploy the new changes to their application.

A few steps are involved in rolling out contract testing, which could seem daunting. However, each step involved within the life cycle in isolation is simple. Much of the complexity related to the contract testing process comes down to communication and collaboration. Imagine how cumbersome the steps described would be without a tool or framework to facilitate the process. For example, sharing contracts between development teams means someone needs to manage access to contracts between potentially many development teams. The contracts would live in source control, where the contracts need to be organized efficiently within the different project repositories. Luckily, this has already been solved by some incredible people at Pact (our thanks to the DiUS team, notably Ron Holshausen, Brent Snook, and Beth Skurrie, for making it happen in 2013 [https://docs.pact.io/history]).

The contract testing life cycle makes much sense when considering the challenges of modern software development, especially in microservices architecture. This type of architecture lends itself to siloed, distributed teams where communication isn't as accessible due to specific barriers. Barriers include physical location, separate backlogs of work, and code